

You Could Have Come Up With Kimi Delta Attention | Doubleword

Jamie Dborin

25~31분

Math notation

*A note on notation: this article defaults to bra-ket notation because (in my quantum-inspired opinion) it makes the shapes in this derivation very clear. The **Math notation** switch above rewrites every equation using conventional bold vectors and explicit transposes instead. In bra-ket mode, $|q\rangle$ is a column vector, $\langle k|$ is a row vector, $\langle k | q \rangle$ is a number, and $|v\rangle\langle k|$ is a matrix. Vectors face right by default, while keys face left when written into the linear-attention state. We work with one causal attention head and real-valued vectors, assume DeltaNet's keys are normalized, and let the state map from key space to value space.*

Modern linear attention variants are complex, and a upon first glance it is not so easy to see what they are designed to achieve. For reference here is the state update equation for Kimi Delta Attention (KDA):

$$\begin{aligned}\tilde{S}_t &= S_{t-1} \text{Diag}(\alpha_t) \\ |\hat{v}_t\rangle &= \tilde{S}_t |k_t\rangle \\ |e_t\rangle &= \beta_t (|v_t\rangle - |\hat{v}_t\rangle) \\ S_t &= \tilde{S}_t + |e_t\rangle\langle k_t| \\ |o_t\rangle &= S_t (d_k^{-1/2} |q_t\rangle)\end{aligned}$$

The reason they are so difficult to understand is that this is the latest in a family of linear attention variants that have been developed over the last few years and the complexity of them has inevitably ballooned such that from the outside the latest variants appear inaccessible.

In this post we are going to walk through the DeltaNet family of linear attention variants, two of which are used by the latest Qwen and Kimi model families, and show how you might have arrived at the same equations by asserting simple things about your hidden state.

That is the route we will take:

softmax attention $\rightarrow$ linear attention $\rightarrow$ [DeltaNet](#) $\rightarrow$ [Gated DeltaNet](#) $\rightarrow$ [KDA](#)

Only after deriving KDA will we turn to the recurrent and chunkwise Triton programs that execute it.

[1. Begin with quadratic attention](#)

For a query at token t , ordinary causal softmax attention is

$$a_{ti} = \frac{\exp(s\langle k_i | q_t \rangle)}{\sum_{j \leq t} \exp(s\langle k_j | q_t \rangle)}, \quad s = d_k^{-1/2},$$
$$|o_t\rangle = \sum_{i \leq t} a_{ti} |v_i\rangle.$$

Every attention weight is a scalar. It measures the similarity between one key and one query, then softmax turns all of the scores for that query into a distribution. The output is a weighted sum of value vectors.

Over a sequence of length T , there are T^2 key-query pairs. During autoregressive inference we can cache the keys and values instead of recomputing them, but the cache still grows

with the sequence and every new query still has to inspect the entire history.

The obstacle to rearranging this computation is the softmax. Its denominator depends jointly on the current query and every earlier key. So, for the moment, remove it.

[1.1 Remove the softmax](#)

For clarity, absorb the constant scale s into the query. The deliberately bare version of attention is then

$$|o_t\rangle = \sum_{i \leq t} \langle k_i | q_t \rangle |v_i\rangle.$$

The scalar inner product can move to the right:

$$\begin{aligned} |o_t\rangle &= \sum_{i \leq t} |v_i\rangle \langle k_i | q_t \rangle \\ &= \left(\sum_{i \leq t} |v_i\rangle \langle k_i | \right) |q_t\rangle. \end{aligned}$$

Everything that depends on the past can now be collected into one matrix of a fixed size $V \times K$:

$$S_t = \sum_{i \leq t} |v_i\rangle \langle k_i|$$

and attention becomes a recurrent write followed by a read:

$$\begin{aligned} S_t &= S_{t-1} + |v_t\rangle \langle k_t|, \\ |o_t\rangle &= S_t |q_t\rangle. \end{aligned}$$

The identity

$$(|v\rangle \langle k|) |q\rangle = \langle k | q \rangle |v\rangle$$

is the whole trick. The outer product is a matrix; the inner product is a number. We no longer store every past key and value. We store their summed outer products in the fixed-size state S_t .

This is linear in sequence length rather than quadratic: scan the tokens once, updating the same $d_v \times d_k$ state at every step. We have paid for that efficiency by discarding softmax's normalization and selectivity. More sophisticated linear-attention methods use feature maps and normalizers, but this unadorned form exposes the memory problem that motivates DeltaNet.

[1.2 Addition is not assignment](#)

Suppose we write a pair $|v_t\rangle\langle k_t|$ and immediately query the new state with that same key:

$$\begin{aligned} S_t | k_t\rangle &= (S_{t-1} + |v_t\rangle\langle k_t|) | k_t\rangle \\ &= S_{t-1} | k_t\rangle + |v_t\rangle \underbrace{\langle k_t | k_t\rangle}_1 \\ &= S_{t-1} | k_t\rangle + |v_t\rangle. \end{aligned}$$

The write does **not** make the memory return $|v_t\rangle$. It adds $|v_t\rangle$ to whatever the memory already returned.

If the old state already produced the correct value, the additive write makes the new state produce twice that value. More generally, keys are not mutually orthogonal, so every write can interfere with previous writes. Linear attention has given us a compact associative memory, but its update behaves like $+=$ when what we want is closer to $=$.

[2. DeltaNet: write the error, not the value](#)

[DeltaNet](#) replaces the unconditional linear-attention write with a delta-rule correction. There are two useful ways to derive it.

[2.1 Derivation one: demand that the write can be read back](#)

Before writing token t , ask the memory what it currently associates with the new key:

$$|\hat{v}_t\rangle = S_{t-1} |k_t\rangle.$$

If we want the memory to return $|v_t\rangle$, we should not add the whole value. We should add only the difference:

$$|v_t\rangle - |\hat{v}_t\rangle.$$

Introduce a learned write strength $\beta_t \in [0, 1]$ and define

$$|e_t\rangle = \beta_t (|v_t\rangle - S_{t-1} |k_t\rangle).$$

Then write this error at the current key:

$$\boxed{S_t = S_{t-1} + |e_t\rangle\langle k_t| .}$$

Now immediately read the same key:

$$\begin{aligned} S_t |k_t\rangle &= S_{t-1} |k_t\rangle + |e_t\rangle\langle k_t|k_t\rangle \\ &= (1 - \beta_t)S_{t-1} |k_t\rangle + \beta_t |v_t\rangle. \end{aligned}$$

When $\beta_t = 1$, the result is exactly $|v_t\rangle$. Smaller β_t moves the old prediction partway towards the target.

The correction is also local in key space. For any query $|x\rangle$ orthogonal to the current key,

$$\langle k_t | x \rangle = 0 \quad \Rightarrow \quad (S_t - S_{t-1}) |x\rangle = |e_t\rangle \underbrace{\langle k_t | x \rangle}_0 = 0.$$

So the rank-one write changes the response in the selected key direction while leaving every orthogonal direction alone.

[2.2 Derivation two: take one step on reconstruction loss](#)

The same update falls out of an online learning objective. Treat the current key-value pair as one training example for the linear map S :

$$\mathcal{L}_t(S) = \frac{1}{2} \|S |k_t\rangle - |v_t\rangle\|_2^2.$$

Its gradient with respect to the state is

$$\nabla_S \mathcal{L}_t(S) = (S |k_t\rangle - |v_t\rangle)\langle k_t| .$$

This is visibly an outer product: a value-space prediction error times the key bra at which that error was observed. Take one gradient-descent step of size β_t from S_{t-1} :

$$\begin{aligned} S_t &= S_{t-1} - \beta_t \nabla_S \mathcal{L}_t(S_{t-1}) \\ &= S_{t-1} - \beta_t (S_{t-1} | k_t\rangle - | v_t\rangle) \langle k_t| \\ &= S_{t-1} + \beta_t (| v_t\rangle - S_{t-1} | k_t\rangle) \langle k_t|. \end{aligned}$$

This is exactly the update we got by requiring immediate reconstruction. The two interpretations are the same:

- as a memory operation, β_t controls how strongly to replace the old association;
- as online learning, β_t is the step size;
- as linear algebra, the change is a rank-one outer product.

[2.3 The DeltaNet state transition](#)

Expanding the error exposes DeltaNet as a structured state transition plus a new input:

$$\begin{aligned} S_t &= S_{t-1} + \beta_t (| v_t\rangle - S_{t-1} | k_t\rangle) \langle k_t| \\ &= S_{t-1} (I - \beta_t | k_t\rangle \langle k_t|) + \beta_t | v_t\rangle \langle k_t|. \end{aligned}$$

For a unit key, $I - \beta_t | k_t\rangle \langle k_t|$ has eigenvalue $1 - \beta_t$ in the current key direction and eigenvalue 1 in every orthogonal direction. It removes the old association along the current key before adding the new one.

DeltaNet fixes the write. It does not yet fix the lifetime of the state.

[3. Gated DeltaNet: sometimes old information should disappear](#)

The linear state compresses the whole history into one matrix. A

read

$$S_t | q \rangle = \sum_{i \leq t} \langle k_i | q \rangle | v_i \rangle$$

cannot choose to skip an individual old token after that token has been folded into S_t . Every stored direction that overlaps the query contributes. The delta rule can correct the state around the current key, but stale information in other directions remains available and can distort future reads.

We therefore need a way to forget the old state before using it. Let $\alpha_t \in [0, 1]$ be a learned scalar retention gate:

$$\tilde{S}_t = \alpha_t S_{t-1}.$$

Run the same delta rule against this gated state:

$\tilde{S}_t = \alpha_t S_{t-1},$	forget,
$ \hat{v}_t\rangle = \tilde{S}_t k_t \rangle,$	predict,
$ e_t\rangle = \beta_t (v_t \rangle - \hat{v}_t \rangle),$	correct,
$S_t = \tilde{S}_t + e_t \rangle \langle k_t ,$	write.

This is [Gated DeltaNet](#). The order matters: forget first, predict from the retained state, then correct that prediction. If we predicted before forgetting, the error would describe a different memory from the one we update.

Expanding the recurrence gives

$$S_t = \alpha_t S_{t-1} (I - \beta_t | k_t \rangle \langle k_t |) + \beta_t | v_t \rangle \langle k_t | .$$

The delta rule gives targeted replacement; the scalar gate gives global erasure. They solve different problems and are complementary.

But α_t still makes one decision for the entire matrix. The model must retain or forget every key channel at the same rate.

[4. Kimi Delta Attention: forget each channel](#)

independently

[Kimi Delta Attention](#) replaces Gated DeltaNet's scalar retention with a vector $\alpha_t \in [0, 1]^{d_k}$. Put the vector on the diagonal:

$$D_t = \text{Diag}(\alpha_t) \in \mathbb{R}^{d_k \times d_k}.$$

Our state maps keys to values, so the key channels are the columns of S . Right-multiplication applies a different retention factor to every one:

$$\tilde{S}_t = S_{t-1} D_t.$$

Everything else is the delta rule we have already derived:

$\tilde{S}_t = S_{t-1} D_t,$	forget each key channel,
$ \hat{v}_t\rangle = \tilde{S}_t k_t\rangle,$	predict,
$ e_t\rangle = \beta_t (v_t\rangle - \hat{v}_t\rangle),$	correct,
$S_t = \tilde{S}_t + e_t\rangle\langle k_t ,$	write,
$ o_t\rangle = S_t (s q_t\rangle), \quad s = d_k^{-1/2},$	read.

That is KDA. Compared with Gated DeltaNet, the conceptual change is only the promotion

$$\alpha_t \rightarrow D_t = \text{Diag}(\alpha_t).$$

The effect is substantial: one channel can be cleared while another is retained.

4.1 Why the transition is diagonal-plus-low-rank

Expand the KDA correction:

$$\begin{aligned} S_t &= S_{t-1} D_t + \beta_t (|v_t\rangle - S_{t-1} D_t |k_t\rangle) \langle k_t| \\ &= S_{t-1} \underbrace{D_t (I - \beta_t |k_t\rangle \langle k_t|)}_{A_t} + \beta_t |v_t\rangle \langle k_t|. \end{aligned}$$

The key-space transition is

$$\begin{aligned} A_t &= D_t - \beta_t D_t |k_t\rangle \langle k_t| \\ &= D_t - |b_t\rangle \langle a_t|, \end{aligned}$$

where

$$|b_t\rangle = D_t |k_t\rangle, \quad \langle a_t| = \beta_t \langle k_t|.$$

So A_t is a diagonal matrix minus a rank-one matrix: a **diagonal-plus-low-rank**, or DPLR, transition. “DPLR” describes the $d_k \times d_k$ transition acting on key space. The memory state itself is still the $d_v \times d_k$ matrix S_t .

The full journey can now be summarized compactly:

Mechanism	State update	What it adds
Linear attention	$S + v\rangle\langle k $	Fixed-size recurrent memory
DeltaNet	$S + \beta(v\rangle - S k\rangle)\langle k $	Targeted replacement
Gated DeltaNet	Apply αS , then the delta update	Whole-state forgetting
KDA	Apply SD , then the delta update	Per-key-channel forgetting

The implementation usually stores $g_t = \log \alpha_t$ with $g_t \leq 0$, then obtains the retention factors as $\exp(g_t)$. In the transposed $d_k \times d_v$ layout used by the reference code, the recurrence is only five lines:

```
state = state * g_t.exp().unsqueeze(-1)
prediction = einsum("bhkv,bhk->bhv", state, k_t)
residual = beta_t.unsqueeze(-1) * (v_t -
prediction)
state = state + einsum("bhk,bhv->bhkv", k_t,
residual)
output = einsum("bhk,bhkv->bhv", q_t * scale,
```

```
state)
```

See the official [naive_recurrent_kda](#) reference.

[5. The fused recurrent Triton kernel](#)

The recurrence above is the natural implementation for autoregressive decode. KDA has two principal execution regimes:

Regime	Best use	Parallel unit
Fused recurrent	Decode, short sequences, stateful serving	One sequence, value head, and value tile
Chunkwise	Training and long prefill	Chunks, token subchunks, and key/value tiles

The recurrent Triton launch uses one program per sequence, value head, and 32-wide value tile:

```
BK = triton.next_power_of_2(K)
BV = 32
grid = (triton.cdiv(V, BV) * N * HV,)
```

See the [fused_recurrent_kda_fwd](#) launch code.

BK covers the key dimension in the normal supported configuration. Each program owns a [BK, BV] tile of the implementation's transposed state and loops over tokens in order. Different value tiles, heads, and sequences run independently.

The kernel is almost a literal transcription of the recurrence:

```
state *= tl.exp(g_t[:, None])
```

```

prediction = tl.sum(state * k_t[:, None], axis=0)
residual = beta_t * (v_t - prediction)
state += k_t[:, None] * residual[None, :]
out_t = tl.sum(state * (q_t * SCALE)[:, None],
axis=0)

```

The prediction and read are reductions; the write is an outer product. This is excellent for decode, where only one new token is available at a time. It is less attractive for training and long prefill because these vector operations do not become the large matrix multiplications on which tensor cores are most efficient.

That motivates a second view of exactly the same recurrence.

[6. Chunkwise KDA](#)

Chunkwise KDA processes C tokens together. It must produce exactly the same states and outputs as the token-by-token recurrence, but it reorganizes the work into matrix products.

For each chunk c , we need two results:

1. the state S_{c+1} after the entire chunk, given the incoming state S_c ;
2. every causal token output inside the chunk.

The only difficulty is that token i 's delta error depends on writes made by earlier tokens in the same chunk. A four-token example makes those dependencies explicit.

[6.1 Decay notation for a four-token chunk](#)

Take tokens 0, 1, 2, 3 and define

$$D_i = \text{Diag}(\alpha_i).$$

The cumulative decay between the chunk boundary and token i is

$$D_{0:i} = D_0 D_1 \cdots D_i.$$

The decay carrying a write at token j forward to token i is

$$D_{j+1:i} = D_{j+1} D_{j+2} \cdots D_i, \quad j < i,$$

with $D_{i+1:i} = I$ when no intervening decay exists. All of these matrices are diagonal, so they commute with one another.

[6.2 Start with provisional errors](#)

First pretend that each token can see the appropriately decayed incoming state but none of the other writes inside its chunk:

$$|\bar{e}_i\rangle = \beta_i (|v_i\rangle - S_c D_{0:i} |k_i\rangle).$$

For four tokens this gives four provisional value-space error kets:

$$|\bar{e}_0\rangle, \quad |\bar{e}_1\rangle, \quad |\bar{e}_2\rangle, \quad |\bar{e}_3\rangle.$$

They are easy to compute in parallel, but all except the first are wrong: earlier writes in the same chunk also contribute to their predictions.

[6.3 Restore the causal dependencies](#)

Token 0 has no earlier in-chunk write, so

$$|e_0\rangle = |\bar{e}_0\rangle.$$

Token 1 sees token 0's write after it has passed through D_1 :

$$|e_1\rangle = |\bar{e}_1\rangle - \beta_1 \langle k_0 | D_1 | k_1 \rangle |e_0\rangle.$$

Token 2 sees both preceding writes:

$$\begin{aligned} |e_2\rangle = & |\bar{e}_2\rangle \\ & - \beta_2 \langle k_0 | D_1 D_2 | k_2 \rangle |e_0\rangle \\ & - \beta_2 \langle k_1 | D_2 | k_2 \rangle |e_1\rangle. \end{aligned}$$

Token 3 sees all three:

$$\begin{aligned}
|e_3\rangle &= |\bar{e}_3\rangle \\
&\quad - \beta_3 \langle k_0 | D_1 D_2 D_3 | k_3 \rangle |e_0\rangle \\
&\quad - \beta_3 \langle k_1 | D_2 D_3 | k_3 \rangle |e_1\rangle \\
&\quad - \beta_3 \langle k_2 | D_3 | k_3 \rangle |e_2\rangle.
\end{aligned}$$

Every bracket $\langle k_j | D_{j+1:i} | k_i \rangle$ is a scalar. Define the causal key-key coefficient

$$\boxed{\rho_{ij} = \beta_i \langle k_j | D_{j+1:i} | k_i \rangle, \quad j < i.}$$

Then all four equations have the compact form

$$\begin{aligned}
|e_0\rangle &= |\bar{e}_0\rangle, \\
|e_1\rangle &= |\bar{e}_1\rangle - \rho_{10} |e_0\rangle, \\
|e_2\rangle &= |\bar{e}_2\rangle - \rho_{20} |e_0\rangle - \rho_{21} |e_1\rangle, \\
|e_3\rangle &= |\bar{e}_3\rangle - \rho_{30} |e_0\rangle - \rho_{31} |e_1\rangle - \rho_{32} |e_2\rangle.
\end{aligned}$$

Collect the coefficients into a strictly lower-triangular matrix:

$$R_c = \begin{bmatrix} 0 & 0 & 0 & 0 \\ \rho_{10} & 0 & 0 & 0 \\ \rho_{20} & \rho_{21} & 0 & 0 \\ \rho_{30} & \rho_{31} & \rho_{32} & 0 \end{bmatrix}, \quad A_c^{kk} = (I + R_c)^{-1}.$$

Stack the error kets as columns:

$$\bar{E}_c = [|\bar{e}_0\rangle \quad |\bar{e}_1\rangle \quad |\bar{e}_2\rangle \quad |\bar{e}_3\rangle],$$

and likewise for E_c . The causal substitutions are then

$$\boxed{E_c = \bar{E}_c (A_c^{kk})^\top.}$$

The implementation does not need to form a general dense inverse. Because $I + R_c$ is triangular with ones on its diagonal, the operation is a causal triangular solve, applied independently to every value channel.

[6.4 Fast-forward the state](#)

At the end of the chunk, the incoming state has passed through

all four decays. Each in-chunk write has passed through only the decays after it:

$$\begin{aligned}
S_{c+1} = & S_c D_0 D_1 D_2 D_3 \\
& + |e_0\rangle\langle k_0| D_1 D_2 D_3 \\
& + |e_1\rangle\langle k_1| D_2 D_3 \\
& + |e_2\rangle\langle k_2| D_3 \\
& + |e_3\rangle\langle k_3|.
\end{aligned}$$

Define the matrix whose rows are the keys as they arrive at the end boundary:

$$K_c^{\text{end}} = \begin{bmatrix} \langle k_0 | D_1 D_2 D_3 \\ \langle k_1 | D_2 D_3 \\ \langle k_2 | D_3 \\ \langle k_3 | \end{bmatrix}.$$

Because E_c stacks the error kets as columns, all four outer-product writes become one matrix multiplication:

$$\boxed{S_{c+1} = S_c D_{0:3} + E_c K_c^{\text{end}}.}$$

This is the first required chunk result: advance the recurrent state by four tokens at once.

[6.5 Compute every causal output](#)

KDA reads after writing. If $S^{[i+1]}$ is the local state after token i , then

$$|o_i\rangle = s S^{[i+1]} |q_i\rangle.$$

Expand the four outputs:

$$\begin{aligned}
|o_0\rangle &= sS_c D_0 |q_0\rangle + s\langle k_0 | q_0\rangle |e_0\rangle, \\
|o_1\rangle &= sS_c D_0 D_1 |q_1\rangle \\
&\quad + s\langle k_0 | D_1 | q_1\rangle |e_0\rangle + s\langle k_1 | q_1\rangle |e_1\rangle, \\
|o_2\rangle &= sS_c D_0 D_1 D_2 |q_2\rangle \\
&\quad + s\langle k_0 | D_1 D_2 | q_2\rangle |e_0\rangle \\
&\quad + s\langle k_1 | D_2 | q_2\rangle |e_1\rangle + s\langle k_2 | q_2\rangle |e_2\rangle, \\
|o_3\rangle &= sS_c D_0 D_1 D_2 D_3 |q_3\rangle \\
&\quad + s\langle k_0 | D_1 D_2 D_3 | q_3\rangle |e_0\rangle \\
&\quad + s\langle k_1 | D_2 D_3 | q_3\rangle |e_1\rangle \\
&\quad + s\langle k_2 | D_3 | q_3\rangle |e_2\rangle + s\langle k_3 | q_3\rangle |e_3\rangle.
\end{aligned}$$

Define the causal query-key coefficient

$$\boxed{\chi_{ij} = s\langle k_j | D_{j+1:i} | q_i\rangle, \quad j \leq i,}$$

and place the coefficients in a lower-triangular read matrix:

$$A_c^{qk} = \begin{bmatrix} \chi_{00} & 0 & 0 & 0 \\ \chi_{10} & \chi_{11} & 0 & 0 \\ \chi_{20} & \chi_{21} & \chi_{22} & 0 \\ \chi_{30} & \chi_{31} & \chi_{32} & \chi_{33} \end{bmatrix}.$$

The zeros enforce causality. The diagonal is included because token i reads after making its own write.

Now stack the boundary-decayed query kets as columns,

$$Q_c^{\text{boundary}} = [D_0 | q_0\rangle \quad D_0 D_1 | q_1\rangle \quad D_0 D_1 D_2 | q_2\rangle \quad D_0 D_1 D_2 D_3 | q_3\rangle],$$

and stack the output kets the same way. All four outputs are

$$\boxed{O_c = sS_c Q_c^{\text{boundary}} + E_c (A_c^{qk})^T.}$$

The first matrix product reads the appropriately decayed incoming state. The second adds the causal contribution of writes made inside the chunk. This is the second required chunk result.

[6.6 How the Triton pipeline is organized](#)

The chunkwise implementation turns the equations above into a pipeline of kernel launches rather than one monolithic kernel.

It first computes chunk-local cumulative log decays. Differences between two prefix sums encode $D_{j+1:i}$ without explicitly multiplying a long chain of retention vectors. It then constructs the causal A^{qk} and A^{kk} interaction matrices. A^{kk} is used to form a WY-style representation of the chunk's corrected writes.

A state kernel performs the only inter-chunk scan, producing the state entering each chunk and resolving its delta errors. Once those incoming states are known, an output kernel can calculate the tokens in different chunks and tiles in parallel.

The real source contains fused and tiled variants of these stages. In particular, it computes 16-token diagonal interaction blocks before a fused off-diagonal and triangular-solve kernel. The forward dataflow is approximately:

```
def chunkwise_kda(q, k, v, log_decay, beta,
initial_state, scale):
    # Within-chunk prefix sums. G[i] - G[j] encodes
the decay
    # carrying a state or write from token j to
token i.
    G = chunk_local_cumsum(log_decay)

    # Build causal query-key interactions and the
triangular system
    # that resolves dependencies between delta
errors.
    A_qk_diag, A_kk_diag = intra_token_parallel(
        q, k, G, beta, scale
    )
```



```

    A_qk, A_kk = inter_and_triangular_solve(
        q, k, G, beta, A_qk_diag, A_kk_diag, scale
    )

    # Convert the chunk into its WY pseudo-key/
    pseudo-value form.
    W, U, K_to_end = build_wy_factors(k, v, G,
    beta, A_kk)

    # The only recurrence left is over chunk
    boundaries.
    H, E, final_state = scan_chunk_states(
        K_to_end, W, U, G, initial_state
    )

    # Combine reads from each incoming chunk state
    with causal
    # contributions from writes made inside that
    chunk.
    output = calculate_outputs(q, E, G, A_qk, H,
    scale)
    return output, final_state

```

In the reference source these stages are orchestrated by [chunk_kda_fwd](#). Its principal implementation entry points are `chunk_kda_fwd_intra`, `chunk_gated_delta_rule_fwd_h`, and `chunk_gla_fwd_o_gk`. Names such as `v_new`, `h`, and `kg` in the code correspond to the resolved errors, incoming chunk states, and keys decayed to the end of a chunk.

The recurrent and chunkwise programs are therefore not two different attention mechanisms. They are two schedules for the same KDA recurrence: serial vector operations for low-latency

decode, and chunked matrix operations for tensor-core-heavy training and prefill.